



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
مدل‌های مولد عمیق
دکتر توسلی‌پور
تمرین سوم

نام و نام خانوادگی	علیرضا نجفی مطیعی
شماره دانشجویی	۸۱۰۱۰۰۲۲۴
تاریخ ارسال گزارش	۱۴۰۴.۱۰.۰۷

فهرست

۳.....	سؤال اول: Energy-Based Model
۳.....	بخش اول: بخش تئوری
۵.....	بخش دوم: بخش پیاده‌سازی
۱۴.....	سوال دوم: Score-Based Models
۱۴.....	بخش اول: سوالات تئوری
۱۹.....	بخش دوم: سوالات پیاده‌سازی

سؤال اول: Energy-Based Model

بخش اول: بخش تئوری

زیربخش اول:

$$P(x) = \frac{\exp(-E(x))}{Z}$$

در اینجا محاسبه Z بسیار دشوار و گاهی غیر ممکن است چرا که باید یکسری انتگرال پیچیده را محاسبه کنیم.

در آموزش یک مدل مبتنی بر انرژی چالش اصلی همان محاسبه Z (Partition Function) است که بسیار دشوار و زمانبر است. برای آموزش راحت تر و پایدارتر نیاز است تا نیازی به Z نباشد. برای این منظور می توانیم از روش های Score Matching و بویژه Denoising Score Matching استفاده کنیم. در روش های سنتی ما سعی می کنیم $P(x)$ را به حداکثر برسانیم که برای این کار نیاز است تا Z را داشته باشیم. اما در روش Score Matching تلاش است تا گرادیان یا شیب تابع انرژی را یاد بگیریم. درواقع $\nabla_x \log(p(x))$ را یاد می گیریم که به آن Score Function می گویند. در اینجا چون Z نسبت به x ثابت است در زمان مشتق گرفتن حذف می شود و بنابراین دیگر نیازی به محاسبه Z نداریم.

پس از آن که مدل با یک از روش های Score Matching آموزش دید از Langevin Dynamics برای تولید تصویر استفاده می کنیم. به اینگونه که از یک نویز تصادفی شروع می کنیم سپس در هر مرحله تصویر را کمی در جهت گرادیان که انرژی را کاهش می دهد حرکت می دهیم تا نهایتا به تصویر واقعی و با انرژی پایین برسیم.

زیربخش دوم:

در اکثر مواقع ما قصد داریم از یک توزیع پیچیده هدف $P(x)$ نمونه برداری کنیم اما انجام این کار به صورت مستقیم دشوار است بنابراین از یک توزیع پیشنهادی ساده تر $Q(x)$ مثل توزیع گاوسی که نمونه برداری از آن آسان است استفاده می کنیم. به این صورت عمل می کنیم که ابتدا یک k ثابت پیدا می کنیم به طوریکه برای تمام x ها داشته باشیم: $Q(x) \geq \tilde{P}(x)$ یعنی kQ باید کل توزیع هدف را بپوشاند. پس از آن یک نمونه x از توزیع پیشنهادی $Q(x)$ تولید می کنیم. سپس یک عدد تصادفی u از بازه $[0,1]$ تولید می کنیم. اگر $u \leq \frac{\tilde{P}(x)}{kQ(x)}$ باشد نمونه x را می پذیریم. در غیر این صورت نمونه را رد و دوباره تکرار می کنیم. احتمال اینکه یک نمونه تولید شده پذیرفته شود برابر است با $\frac{1}{k}$ با فرض نرمال بودن توزیع ها. هرچه k بزرگتر باشد، یعنی فاصله توزیع پیشنهادی و هدف زیاد باشد، نرخ پذیرش پایین می آید و الگوریتم کند می شود.

ابعاد داده‌های MNIST برابر است با $28 \times 28 = 784 = D$.

$$k = \max_x \frac{P^{\sim}(x)}{Q(x)}$$

$$\frac{P^{\sim}(x)}{Q(x)} = \frac{\left((2\pi\sigma_p^2)^{-\frac{D}{2}} \exp\left(-\frac{\|x - \mu\|^2}{2\sigma_p^2}\right) \right)}{\left((2\pi\sigma_q^2)^{-\frac{D}{2}} \exp\left(-\frac{\|x - \mu\|^2}{2\sigma_q^2}\right) \right)}$$

$$\rightarrow \frac{P^{\sim}(x)}{Q(x)} = \left(\frac{\sigma_q}{\sigma_p}\right)^D \exp\left(-\frac{\|x - \mu\|^2}{2} \left(\frac{1}{\sigma_p^2} - \frac{1}{\sigma_q^2}\right)\right)$$

چون $\sigma_q > \sigma_p$ عبارت $\left(\frac{1}{\sigma_p^2} - \frac{1}{\sigma_q^2}\right)$ مثبت است. بنابراین توان کلی منفی است. برای ماکزیمم کردن باید توان نمایی را به صفر برسانیم که در $x = \mu$ اتفاق می‌افتد. بنابراین:

$$k = \left(\frac{\sigma_q}{\sigma_p}\right)^D$$

$$\rightarrow k = (1.01)^{784}$$

$$Acceptance\ rate = \frac{1}{k} = (0.99)^{784} \approx 0.0004 = 0.04\%$$

یعنی با اینکه توزیع Q را تنها ۱ درصد پهن‌تر از توزیع هدف انتخاب کردیم اما به دلیل ابعاد بالای تصویر نرخ پذیرش تقریباً به صفر میل می‌کند. یعنی باید حدوداً ۲۵۰۰ بار نمونه‌گیری انجام شود تا یک نمونه قابل قبول بدست بیاید. این مسائل نشان‌دهنده این است که rejection sampling برای مدل‌های جنریتو در ابعاد بالا تقریباً غیر قابل استفاده است و باید از روش‌های پیشرفته‌تر مثل MCMC استفاده کرد.

زیربخش سوم:

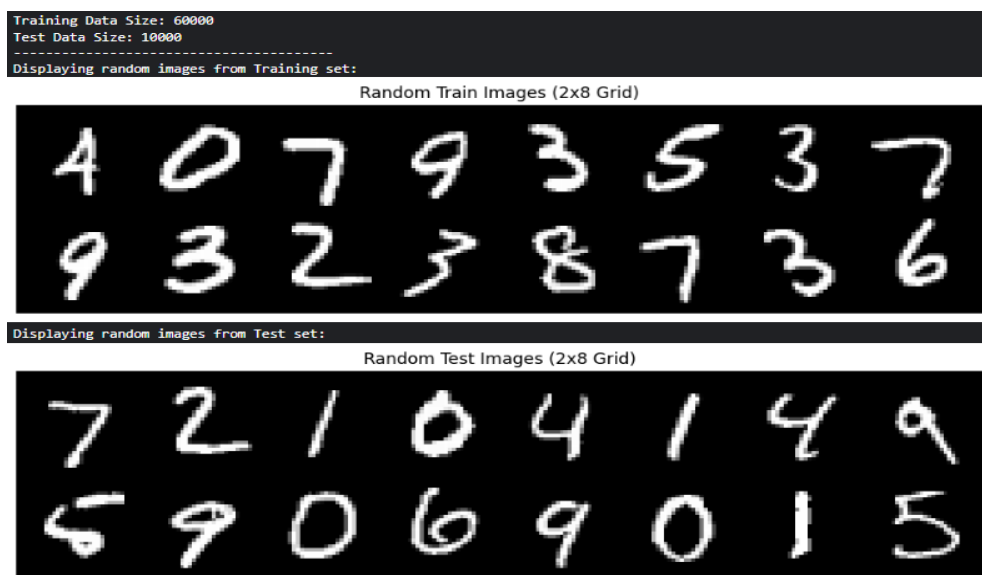
الف) هدف آموزش به روش Maximum Likelihood افزایش احتمال داده‌های مشاهده شده است. با توجه به درنظر گرفتن علامت‌ها در گرادیان برای داده‌های آموزشی یا x_{train} این ترم علامت مثبت دارد بنابراین فرآیند آموزش سعی می‌کند مقدار f_{θ} (که با توجه به نوع نوشتار صورت پروژه و اسلاید درس در اینجا تابع امتیاز یا انرژی منفی است و نه خود انرژی) را برای داده‌های واقعی افزایش دهد. یعنی انرژی را که در اینجا منفی f_{θ} است را کم کند تا احتمالشان زیاد شود. برای نمونه‌های تولیدی x_{sample} ترم دوم علامت منفی دارد بنابراین آموزش سعی می‌کند مقدار f_{θ} را برای نمونه‌هایی که اکنون توسط مدل تولید می‌شوند و احتمال بالا هنوز واقعی نیستند را کاهش دهد. به طور کلی یعنی مدل سعی می‌کند برای داده‌های واقعی پاداش درنظر بگیرد و نمونه‌های فعلی را به نوعی تنبیه کند تا آنجایی که توزیع مدل بر توزیع داده‌ها منطبق شود.

ب) در ترم شامل امیدریاضی برای محاسبه دقیق این ترم باید از توزیع فعلی نمونه برداری کنیم و از آنجایی که ثابت نرمالسازی Z را نداریم نمونه برداری به صورت مستقیم غیر ممکن است و باید از روش‌های MCMC استفاده کنیم. این روش‌ها برای رسیدن به همگرایی نیاز به تکرارهای بسیار زیاد دارند و انجام یک فرآیند طولانی MCMC در هر گام از آموزش از نظر محاسباتی بسیار سنگین یا حتی غیر ممکن است. در روش CD به جای اینکه زنجیره مارکوف را تا بی‌نهایت یا یک زمان طولانی برای همگرایی ادامه دهیم آن را فقط برای k مرحله کوتاه اجرا می‌کنیم (Truncated MCMC). همچنین به جای اینکه زنجیره را از نویز تصادفی آغاز کنیم آن را از داده‌های آموزشی واقعی شروع می‌کنیم. ایده کلی CD این است که اگر مدل خوب باشد داده‌های واقعی باید در نقاط با احتمال بالا باشند و زنجیره نباید از آن‌ها دور شود و اگر مدل بد باشد زنجیره از داده واقعی دور می‌شود و به سمت نقاطی می‌رود که مدل به اشتباه احتمال بالایی به آن‌ها داده است. روش سعی دارد تا همین نقاط اشتباه را اصلاح کند.

بخش دوم: بخش پیاده‌سازی

زیربخش اول: بارگذاری دیتاست و آماده‌سازی

پس از بارگذاری دیتاست و اعمال تبدیلات لازم و همچنین تعریف داده‌های train, test, dataloader خروجی‌ها را به نمایش می‌گذاریم.



شکل ۱. تعداد داده‌ها و چند تصویر تصادفی از train و test

زیربخش دوم: پیاده‌سازی معماری و آموزش مدل

مطابق با جدول ۱ و همچنین توضیحات گفته شده معماری را پیاده‌سازی می‌کنیم که خلاصه‌ای از معماری پیاده‌شده در زیر آمده است:

```
=====
MODEL SUMMARY
=====
```

Layer (type:depth-idx)	Input Shape	Output Shape	Param #
EnergyModel	[1, 1, 28, 28]	[1, 1]	--
└Sequential: 1-1	[1, 1, 28, 28]	[1, 1]	--
└└Conv2d: 2-1	[1, 1, 28, 28]	[1, 32, 28, 28]	320
└└└LeakyReLU: 2-2	[1, 32, 28, 28]	[1, 32, 28, 28]	--
└└└Conv2d: 2-3	[1, 32, 28, 28]	[1, 32, 14, 14]	16,416
└└└└LeakyReLU: 2-4	[1, 32, 14, 14]	[1, 32, 14, 14]	--
└└└└Conv2d: 2-5	[1, 32, 14, 14]	[1, 64, 14, 14]	18,496
└└└└└LeakyReLU: 2-6	[1, 64, 14, 14]	[1, 64, 14, 14]	--
└└└└└Conv2d: 2-7	[1, 64, 14, 14]	[1, 64, 7, 7]	65,600
└└└└└└LeakyReLU: 2-8	[1, 64, 7, 7]	[1, 64, 7, 7]	--
└└└└└└Conv2d: 2-9	[1, 64, 7, 7]	[1, 128, 7, 7]	73,856
└└└└└└└LeakyReLU: 2-10	[1, 128, 7, 7]	[1, 128, 7, 7]	--
└└└└└└└AdaptiveAvgPool2d: 2-11	[1, 128, 7, 7]	[1, 128, 1, 1]	--
└└└└└└└└Flatten: 2-12	[1, 128, 1, 1]	[1, 128]	--
└└└└└└└└Linear: 2-13	[1, 128]	[1, 1]	129

```
=====
Total params: 174,817
Trainable params: 174,817
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 13.93
=====
Input size (MB): 0.00
Forward/backward pass size (MB): 0.43
Params size (MB): 0.70
Estimated Total Size (MB): 1.13
=====
```

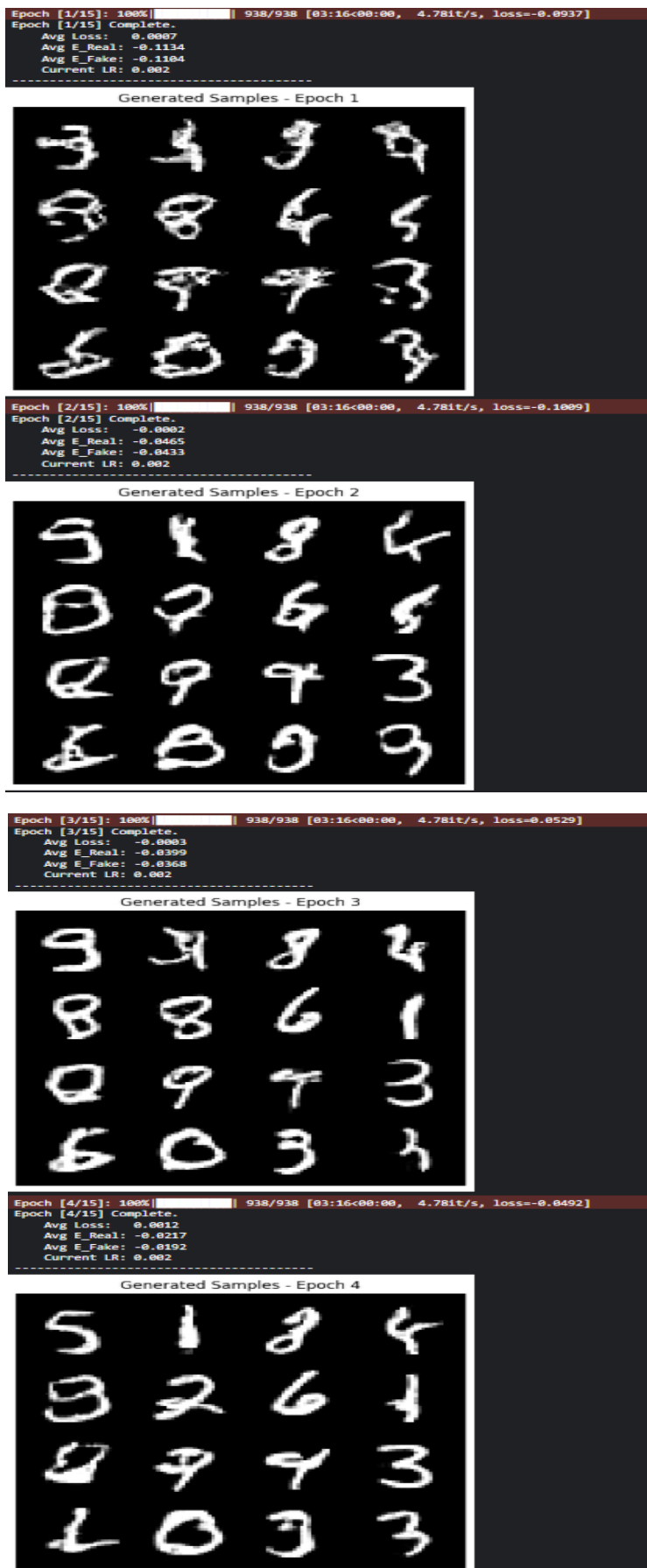
شکل ۲. خلاصه‌ای از معماری پیاده‌سازی شده

تابع Langevin sampling را مطابق با صورت پروژه به گونه‌ای طراحی می‌کنیم که بتوان مقدار x شروع را به آن داد. پس از آن آموزش را با هایپرپارامترهای زیر و مطابق الگوریتم موجود در سودو کد ۱ و طبق تعاریف خطای موجود در صورت پروژه شروع می‌کنیم.

```
BATCH_SIZE = 64
EPOCHS = 15
LAMBDA_REG = 1e-2
LR = 2 * 1e-3
```

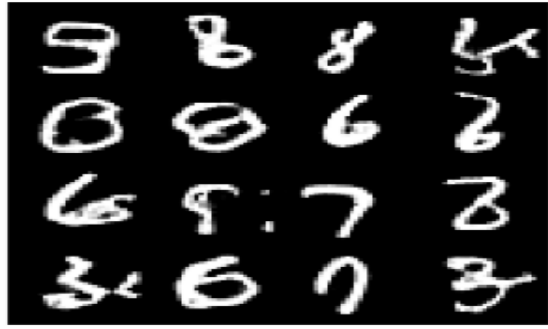
شکل ۳. مقادیر هایپرپارامترها

در اینجا در ۱۵ اپیاک به آموزش مدل می‌پردازیم و لرنینگ ریت را به روش ReduceLROnPlateau قرار می‌دهیم (با $patience$ سه اپیاک) تا در صورت عدم بهبود در یادگیری نرخ یادگیری را نصف کند. همچنین بجای $1e-3$ از لامبدای $1e-2$ استفاده می‌کنیم تا ترم L_{reg} بیش از اندازه کم اهمیت نشود. همچنین مقادیر میانگین E_{real} و E_{fake} و خطای کل در هر اپیاک چاپ شده‌اند. نتایج کلی آموزش به صورت زیر هستند:



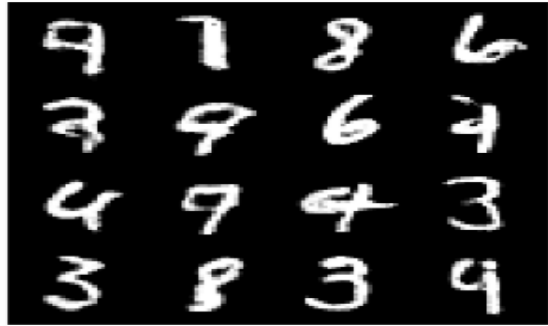
```
Epoch [5/15]: 100% | 938/938 [03:15<00:00, 4.79it/s, loss=-0.0913]
Epoch [5/15] Complete.
Avg Loss: 0.0013
Avg E_Real: -0.0049
Avg E_Fake: -0.0025
Current LR: 0.002
```

Generated Samples - Epoch 5



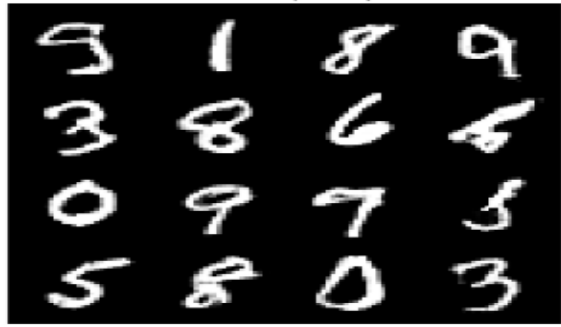
```
Epoch [6/15]: 100% | 938/938 [03:16<00:00, 4.78it/s, loss=-0.1475]
Epoch [6/15] Complete.
Avg Loss: 0.0038
Avg E_Real: -0.0323
Avg E_Fake: -0.0297
Current LR: 0.002
```

Generated Samples - Epoch 6



```
Epoch [7/15]: 100% | 938/938 [03:16<00:00, 4.78it/s, loss=-0.1491]
Epoch [7/15] Complete.
Avg Loss: 0.0018
Avg E_Real: -0.0004
Avg E_Fake: 0.0034
Current LR: 0.001
```

Generated Samples - Epoch 7



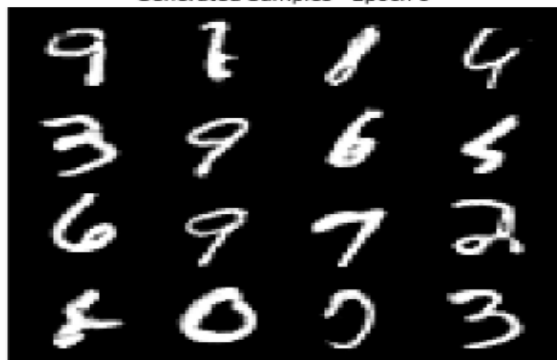
```
Epoch [8/15]: 100% | 938/938 [03:16<00:00, 4.78it/s, loss=-0.0298]
Epoch [8/15] Complete.
Avg Loss: -0.0021
Avg E_Real: -0.0060
Avg E_Fake: 0.0017
Current LR: 0.001
```

Generated Samples - Epoch 8



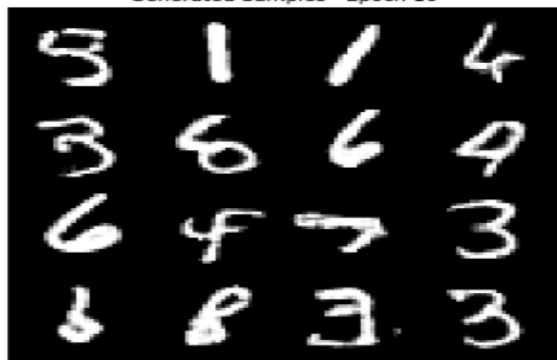
```
Epoch [9/15]: 100%|██████████| 938/938 [03:16<00:00, 4.78it/s, loss=-0.1686]
Epoch [9/15] Complete.
Avg Loss: -0.0040
Avg E_Real: -0.0019
Avg E_Fake: 0.0074
Current LR: 0.001
```

Generated Samples - Epoch 9



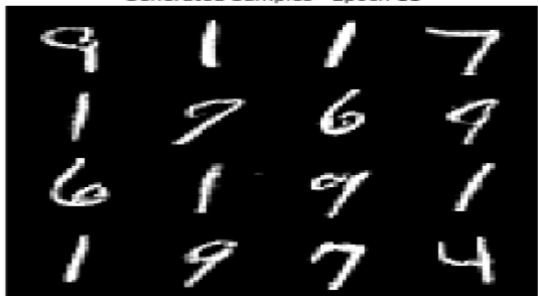
```
Epoch [10/15]: 100%|██████████| 938/938 [03:16<00:00, 4.78it/s, loss=-0.1230]
Epoch [10/15] Complete.
Avg Loss: 0.0002
Avg E_Real: 0.0067
Avg E_Fake: 0.0119
Current LR: 0.001
```

Generated Samples - Epoch 10



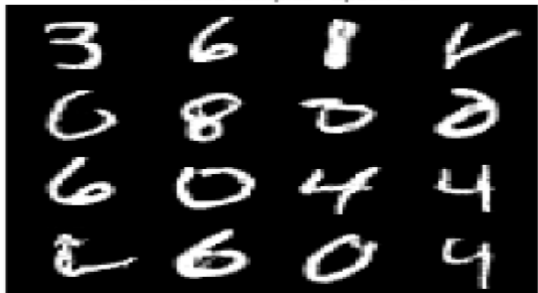
```
Epoch [11/15]: 100%|██████████| 938/938 [03:16<00:00, 4.78it/s, loss=-0.4505]
Epoch [11/15] Complete.
Avg Loss: -0.0011
Avg E_Real: -0.0097
Avg E_Fake: -0.0033
Current LR: 0.001
```

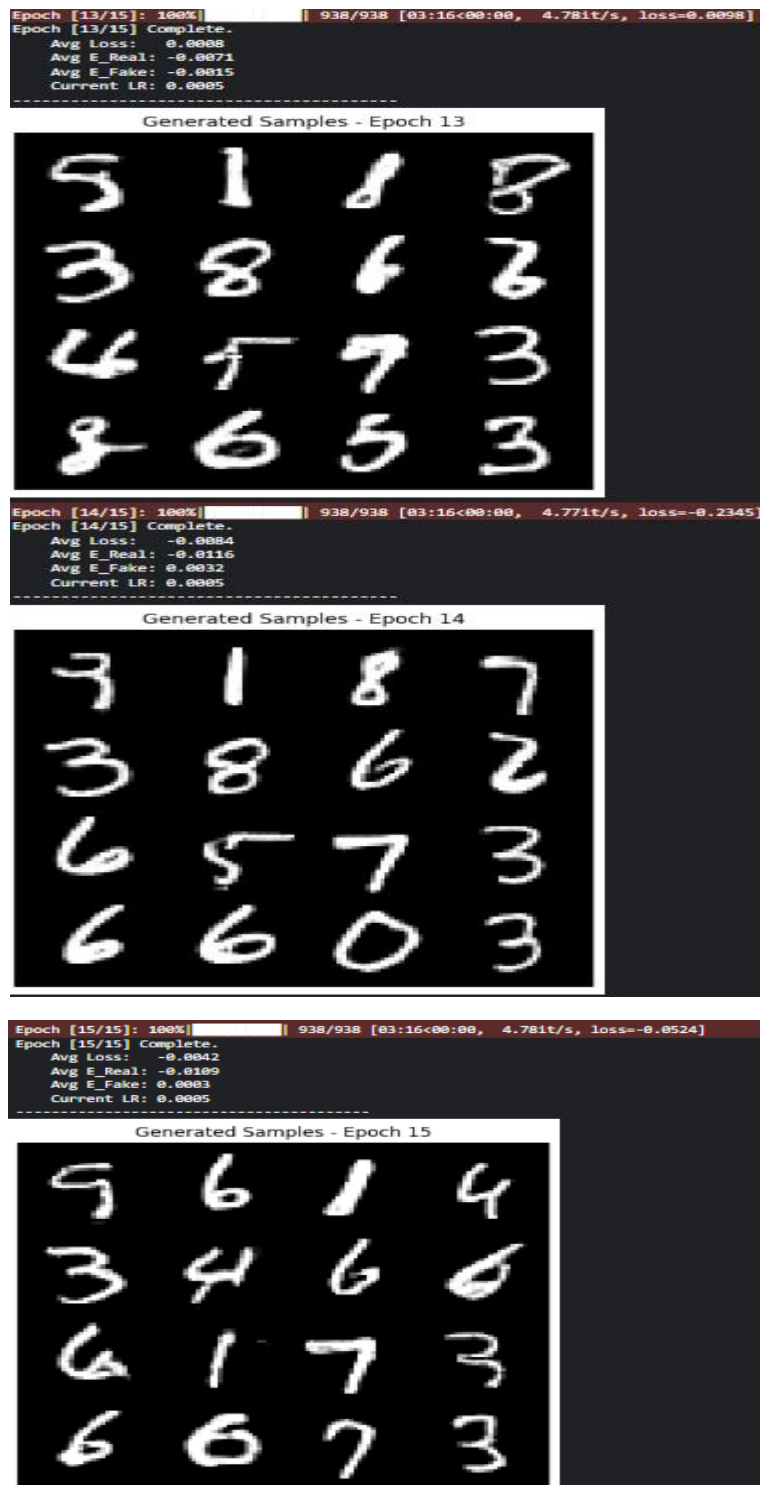
Generated Samples - Epoch 11



```
Epoch [12/15]: 100%|██████████| 938/938 [03:16<00:00, 4.78it/s, loss=-0.0982]
Epoch [12/15] Complete.
Avg Loss: -0.0014
Avg E_Real: 0.0035
Avg E_Fake: 0.0108
Current LR: 0.001
```

Generated Samples - Epoch 12

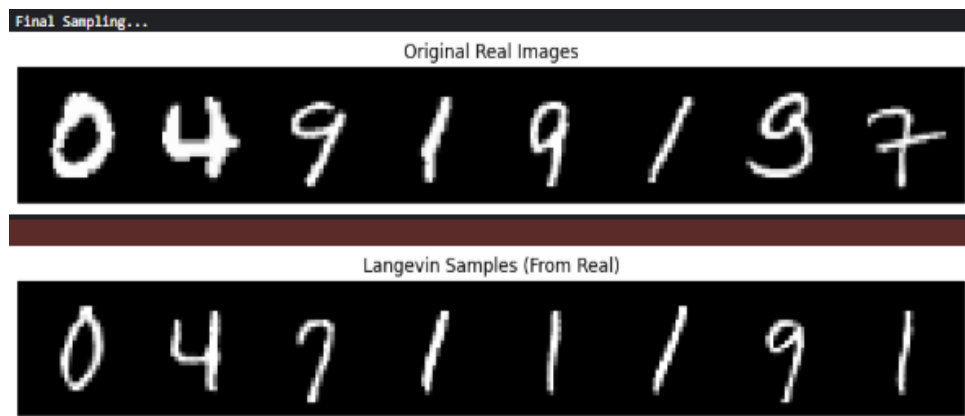




شکل ۴. نتایج خطاهای خواسته شده برای هر ایپاک و تصاویر ایجادشده با Langevin Sampling

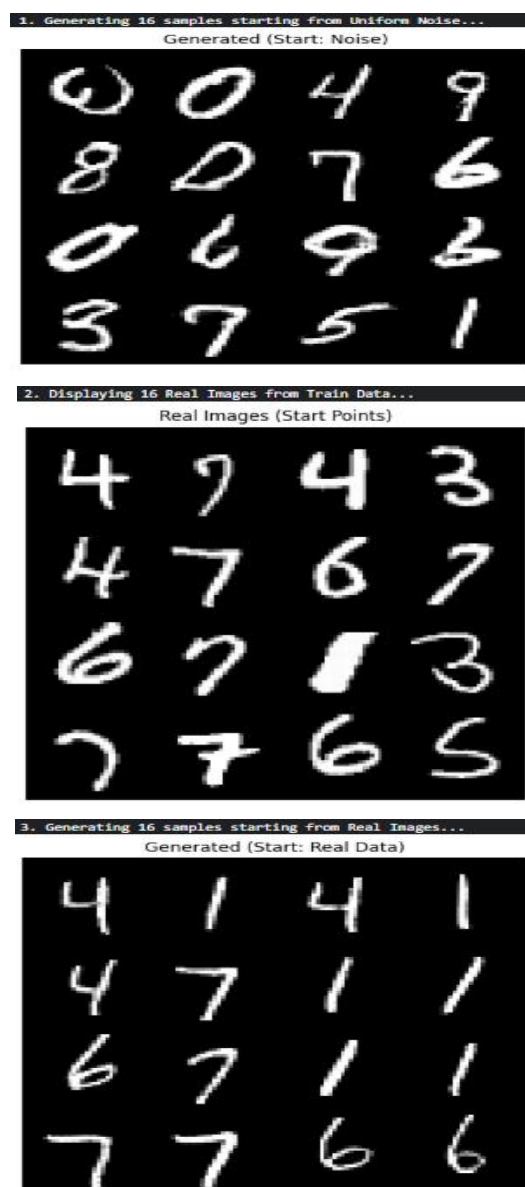
* لاس قرمز نوشته شده مقدار لاس در استپ آخر هر ایپاک است که با خطای میانگین آن ایپاک متفاوت است.

حال که آموزش مدل به پایان رسیده مطابق خواسته سوال تعدادی از تصاویر train را برمی داریم و آن ها را به عنوان نقطه شروع به Langevin sampling می دهیم.



شکل ۵. نمونه‌هایی از تصاویر train و تصاویر تولیدشده

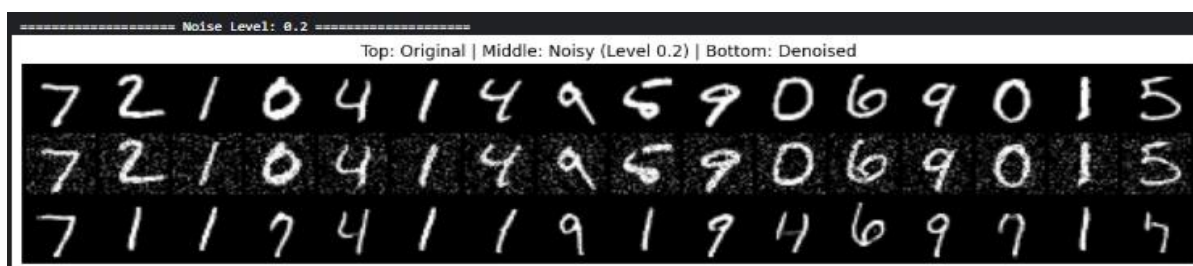
زیربخش سوم: تولید تصویر



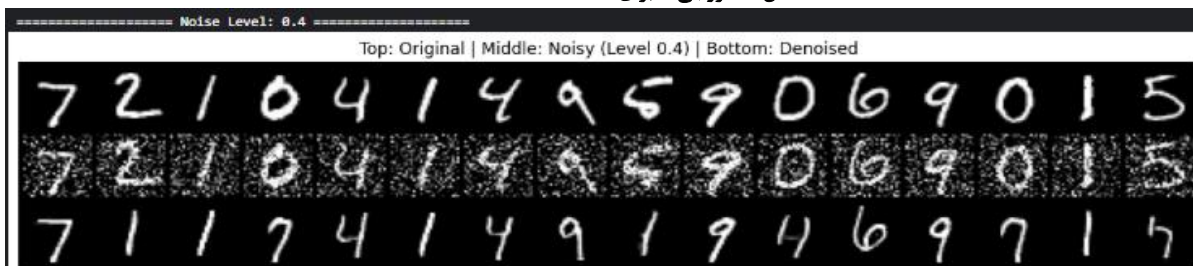
شکل ۶. تصاویر تولیدشده

همانطور که مشاهده می‌شود تصاویر تولیدشده از نویز کیفیت پایینی دارند. اگرچه شباهت‌هایی به اعداد دارند اما اکثرا دارای خطوط شکسته نویزهای پراکنده و ساختاری تقریبا غیرپیوسته هستند و بیشتر شبیه یکسری طرح‌های اولیه هستند تا یکسری اعداد واقعی. خروجی از اعداد واقعی نیز بگونه‌ای هستند که نشان می‌دهد مدل نتوانسته به خوبی از یک داده به داده دیگر حرکت کند. علت اصلی عملکرد ضعیف مشکل نواحی کم‌چگالی و یادگیری ناقص تابع امتیاز است. مشکل در شروع از نویز این است که شبکه عصبی امتیاز تنها روی داده‌های واقعی آموزش دیده است. فضای برداری ابعاد بسیار زیادی دارد و داده‌های واقعی تنها بخش بسیار کوچکی از این فضا یعنی یک منیفولد باریک را اشغال کرده‌اند. وقتی نمونه‌برداری را از نویز یکنواخت شروع می‌کنیم، نقطه شروع در جایی قرار دارد که مدل قبلا هیچ داده‌ای در آن ندیده است. در این نواحی خالی تخمین مدل از جهت گرادینان بسیار نادقیق یا تصادفی است و نمی‌تواند نمونه‌ها را به درستی به سمت منیفولد داده‌ها هدایت کند. همچنین در شروع از داده‌های واقعی توزیع داده‌ها دارای قله‌ها یا مدهای جدا از هم است مثلا قله عدد ۱ جدا از قله عدد ۲ است. بین این قله‌ها نواحی با احتمال بسیار کم قرار دارد. در روش استاندارد Langevin Dynamic مدل نمی‌تواند از این نواحی کم احتمال عبور کند. به همین دلیل وقتی اط یک عکس واقعی شروع می‌کنیم الگورتیم در همان ناحیه گیر می‌کند و نمی‌تواند تصویر متفاوتی تولید کند. راه حل این مشکل استفاده از روش‌هایی مثل Denoising Score Matching است. با افزودن نویز به داده‌های آموزشی عملا نواحی کم‌چگالی با داده‌های نویزی پر می‌شوند. این کار باعث می‌شود تا مدل یاد بگیرد حتی در نقاط دور از داده‌های اصلی نیز چگونه مسیر برگشت را پیدا کند و گرادینان‌ها را در تمام فضا به درستی تخمین بزند.

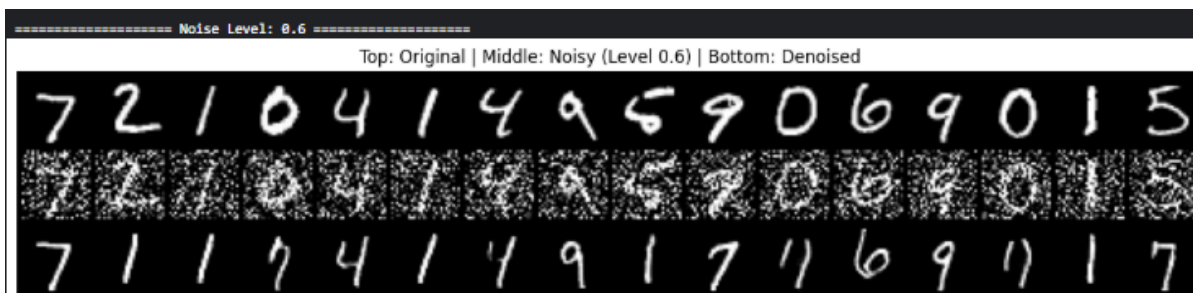
زیربخش چهارم: نویزدایی تصویر نویزی



شکل ۷. خروجی‌ها برای noise level=0.2



شکل ۸. خروجی‌ها برای noise level=0.4



شکل ۹. خروجی‌ها برای noise level=0.6

مشاهده می‌شود که نزدیکی تصویر پردازش‌شده به تصویر اصلی تا حد خوبی به شدت نویز وابسته است اما تصاویر بازسازی شده حتی در نویز کم نیز لزوماً وفاداری کاملی به تصاویر اصلی ندارند. مدل تلاش می‌کند تصویر نویزی را به سمت نزدیک‌ترین نقطه در توزیع داده‌های صحیح هدایت کنند. زمانی که نویز کم است، تصویر نویزی همچنان در نزدیکی تصویر اصلی قرار دارد با این حال مدل تمایل به ساده‌سازی دارد. یعنی یاد گرفته است که تصاویر اعداد باید صاف و تمیز باشند و در فرآیند حذف نویز اگر مدل نسبت به یک جزئیات ظریف مثل انحنای عدد ۲ مطمئن نباشد ترجیح می‌دهد آن را حذف کند و با یک ساختار ساده‌تر مثل عدد ۱ جایگزین کند. نتیجه این است که تصویر خروجی تمیز است و شبیه به یک عدد واقعی می‌باشد اما ممکن است هویت آن نسبت به تصویر اصلی تغییر کرده باشد. زمانی که نویز زیاد است تصویر نویزی از تصویر اصلی بسیار دور می‌شود. مدل تصویر را تمیز می‌کند و نویز را حذف می‌کند اما ممکن است آن را به شکل یک عدد دیگر یا یک نسخه تغییر شکل یافته بازسازی کند. بنابراین در نویزهای بالا اگرچه تصویر خروجی تمیز است اما لزوماً شبیه به تصویر ورودی اصلی نیست. بنابراین مدل در حذف نویز پس‌زمینه در هر سه سطح نویز عملکرد خوبی داشته و توانسته است تا زمینه را سیاه و یکدست کند. همچنین مدل در بازیابی خطوط صاف و اشکال ساده نسبتاً موفق است و این ویژگی‌ها در کل نسبت به نویز مقاوم‌ترند. مهم‌ترین ضعف مدل در جزئیات ظریف منحنی‌های نازک است. که در همان مثال عدد یک و عدد دو مشخص است. مشکل دیگر بیش از حد صاف کردن است یعنی مدل تمایل دارد لبه‌های تیز و زوایای اعداد را گرد و محو کند که این باعث در خروجی تا حدی بافت اصلی خود را از دست بدهند. همچنین در نویزهای بالاتر مدل در تشخیص حفره‌های داخلی اعداد دچار مشکل می‌شود و گاهی یک لکه توپر بازسازی می‌کند. همچنین در نویز بالا مثلاً ۰.۶ مدل در بازیابی شدت روشنایی مشکل دارد و نمی‌تواند روشنایی آن را کامل برگرداند و نتیجه این است که تصویر خروجی کمرنگ می‌شود.

*** به طور کلی با آموزش طولانی‌تر و استفاده از الگوریتم‌های پیچیده‌تر نمونه برداری و بازسازی احتمالاً

نتایج مدل را بهبود دهد.

سوال دوم: Score-Based Models

بخش اول: سوالات تئوری

زیربخش اول:

$$p_{\theta}(x) = \frac{e^{-E_{\theta}(x)}}{Z(\theta)}$$

تابع امتیاز برای توزیع احتمال $p(x)$ به صورت زیر تعریف می‌شود:

$$s(x) = \nabla_x \log p(x)$$

$$\rightarrow \log p_{\theta}(x) = \log \left(\frac{e^{-E_{\theta}(x)}}{Z(\theta)} \right) = \log(e^{-E_{\theta}(x)}) - \log(Z(\theta))$$

$$\nabla_x \log p_{\theta}(x) = \nabla_x (-E_{\theta}(x)) - \nabla_x (\log Z(\theta))$$

در اینجا چون Z فقط به پارامترهای مدل یا θ بستگی دارد و نسبت به ورودی x ثابت است بنابراین مشتق آن نسبت به x صفر می‌شود:

$$\rightarrow s_{\theta}(x) = \nabla_x \log p_{\theta}(x) = -\nabla_x E_{\theta}(x)$$

استقلال از تابع پارتیشن Z به این دلیل دارای مزیت است که محاسبه تابع پارتیشن که انتگرال روی کل فضای ورودی است معمولاً غیر ممکن یا بسیار هزینه بر است. چون تابع امتیاز به Z وابسته نیست می‌توان مدل را بدون نیاز به محاسبه این انتگرال پیچیده آموزش دهیم. این ویژگی باعث می‌شود تا در ادامه بتوانیم از روش‌هایی مانند *score matching* استفاده کنیم.

زیربخش دوم:

تابع امتیاز $s_{\theta}(x)$ یک تابع برداری است که ورودی آن یک x با ابعاد D و خروجی آن نیز یک بردار با بعد D است. tr مجموع عناصر روی قطر اصلی این ماتریس است. برای محاسبه این ترم در یک شبکه عصبی عمیق نیاز است تا مشتق خروجی نام شبکه را نسبت به ورودی نام محاسبه کنیم در روش‌های Backpropagation محاسبه کامل ماتریس ژاکوبی نیازمند انجام عملیات گرادیان گیری به تعداد ابعاد ورودی یعنی D بار است. برای تعداد بعد کم این مشکلی محسوب نمی‌شود اما برای تصاویر با ابعاد بالا باید در هر قدم آموزشی به تعداد ابعاد Backpropagation انجام دهیم.

روش DSM این مشکل را با تغییر در صورت مسئله حل می‌کند. به جای اینکه سعی شود تا مستقیماً مدل را با داده‌های واقعی که گرادیان لگاریتم آن‌ها نامعلوم است تطبیق دهیم، ابتدا داده‌ها را با یک نویز گوسی مشخص ترکیب می‌کنیم و سپس مدل را آموزش می‌دهیم تا نویز را تخمین بزند. اگر داده تمیز x را

با نویز گوسی جمع کنیم تا داده نویزی $x^\sim$ بدست آید، توزیع شرطی $q(x^\sim|x)$ یک توزیع گوسی ساده است. گرادیان لگاریتم در اینجا فرم بسته و ساده دارد:

$$\nabla_x \log q(x^\sim|x) = \frac{x - x^\sim}{\sigma^2} = -\frac{Noise}{\sigma^2}$$

در DSM مدل $s_\theta(x^\sim)$ را آموزش می‌دهیم تا این بردار ساده را تخمین بزنند. تابع هزینه جدید به فرم زیر در خواهد آمد:

$$E_{x, x^\sim} [||s_\theta(x^\sim) - \nabla_x \log q(x^\sim|x)||^2]$$

که این تابع هزینه نیازی به محاسبه ماتریس ژاکوبی ندارد و سریعتر محاسبه می‌شود.

در score matching

$$s_\theta(x^\sim) \approx \nabla_{x^\sim} \log q(x^\sim)$$

که q توزیع حاشیه‌ای است. رابطه بین score شرطی و حاشیه‌ای به صورت زیر است:

$$\nabla_{x^\sim} \log q(x^\sim) = E_{q(x|x^\sim)} [\nabla_{x^\sim} \log q(x^\sim|x)]$$

به عبارت دیگر بهترین برداری که گرادیان کل توزیع مشترک را نشان می‌دهد میانگین تمام بردارهای گرادیان اجزای سازنده آن در آن نقطه است. در روش DSM سعی می‌شود برای هر جفت $x, x^\sim$ خروجی شبکه را به $\nabla_{x^\sim} \log q(x^\sim|x)$ نزدیک شود. وقتی این کار را روی کل دیتاست انجام می‌دهیم و خطای میانگین مربعات را کم می‌کنیم، شبکه عصبی به طور خودکار میانگین اهداف را یاد می‌گیرد یعنی مدل یاد می‌گیرد:

$$s_\theta(x^\sim) \approx E[\nabla_{x^\sim} \log q(x^\sim|x)]$$

که در این رابطه سمت راست score واقعی توزیع نویزی می‌باشد. بنابراین چون بهترین تخمین گر برای خطای میانگین مربعات برابر با میانگین شرطی هدف است، مدل DSM به طور ضمنی گرادیان توزیع نویزی شده را یاد می‌گیرد بدون آنکه نیاز باشد مستقیماً با خود توزیع پیچیده یا ماتریس ژاکوبی درگیر شود.

زیربخش سوم:

(الف)

$$p_{data}(x) = \pi p_1(x) + (1 - \pi) p_2(x)$$

حالت اول: اگر فرض شود که x در ناحیه A است چون در این ناحیه $p_2(x) = 0$ است توزیع به شکل زیر در می‌آید:

$$p_{data}(x) = \pi p_1(x)$$

$$\rightarrow \nabla_x \log p_{data}(x) = \nabla_x (\log \pi) + \nabla_x (\log p_1(x))$$

که عدد پی یک عدد ثابت است. بنابراین:

$$\rightarrow \nabla_x \log p_{data}(x) = \nabla_x (\log p_1(x))$$

حالت دوم: اگر x در ناحیه B باشد چون در این ناحیه $p_1(x) = 0$ است داریم:

$$p_{data}(x) = (1 - \pi)p_2(x)$$

$$\log p_{data}(x) = \log(1 - \pi) + \log p_2(x)$$

که ترم اول نسبت به x ثابت است و حذف می‌شود:

$$\nabla_x \log p_{data}(x) = \nabla_x (\log p_2(x))$$

بنابراین مشاهده شد که در هر دو ناحیه ضریب π در گرادیان گیری حذف شد. تابع امتیاز در ناحیه A تنها شکل p_1 و ناحیه B تنها شکل p_2 را توصیف می‌کند هیچ جمله‌ای در تابع امتیاز نهایی وجود ندارد که بگوید ضریب π چقدر است.

(ب)

الگوریتم Langevin Dynamics برای نمونه‌برداری از گرادیان یا شیب تابع انرژی استفاده می‌کند تا نمونه‌ها را به سمت نواحی با احتمال بالا هدایت کند. ویژگی حذف شدن π دو مشکل اساسی ایجاد می‌کند.

اول اینکه باعث می‌شود تا پدیده Global Weights Blindness اتفاق بیفتد. تابع امتیاز فقط اطلاعات محلی را حفظ می‌کند. یعنی بیان می‌کند که چقدر باید حرکت صورت بگیرد تا به قله توزیع محلی برسیم. اما چون ضریب π حذف شده است این تابع هیچ اطلاعی نمی‌دهد که ارتفاع قله A نسبت به قله B چقدر است. مثلاً اگر این ضریب عدد بالایی باشد الگوریتم باید برای ناحیه A اهمیت بیشتری قائل باشد اما اینگونه نمی‌شود.

مسئله بعدی Lack of Mixing است یعنی توانایی جابجایی بین مدها وجود ندارد. چون دو محل اصلی از هم مجزا هستند یا فاصله میان آن‌ها زیاد است ناحیه بین A و B دارای احتمال صفر یا بسیار پایین است. در این نواحی میانی گرادیان یا تعریف نشده است یا بسیار کوچک است درواقع Vanishing Gradient اتفاق می‌افتد. یعنی اگر نمونه برداری از ناحیه B شروع کنیم الگوریتم در همانجا گیر می‌افتد و راهی برای فهمیدن اینکه ناحیه A هم وجود دارد که شاید هم احتمال بیشتری داشته باشد وجود ندارد. نتیجه این است که نسبت نمونه‌های تولیدشده نهایی بر اساس تابع وزن‌های واقعی نخواهد بود بلکه تابع نقطه شروع اولیه خواهد بود.

زیربخش چهارم:

(الف)

اعمال مستقیم score matching روی داده‌های واقعی با دو چالش اصلی روبرو می‌شود که مانع یادگیری صحیح می‌شوند. اول Manifold Hypothesis یعنی داده‌های واقعی در دنیای واقعی تمایل دارند روی منیفولدهای با ابعاد پایین که در یک فضای با ابعاد بسیار بالا قرار گرفته‌اند متمرکز شوند. تحت این شرایط score matching به مشکلاتی برخورد می‌کند. از نظر ریاضی تابع امتیاز یک گرادیان در کل فضای محیطی است. وقتی داده‌ها محدود به یک منیفولد کم‌بعد باشند این گرادیان در خارج از منیفولد تعریف نشده است. هدف تابع هزینه تنها زمانی می‌تواند یک تخمین زننده سازگار برای تابع امتیاز فراهم کند که support توزیع داده‌ها کل فضا باشد. اگر داده‌ها روی یک منیفولد کم‌بعد باشند این شرط نقض می‌شود و تابع هزینه سازگار نخواهد بود. مشکل بعدی low data density regions است. یعنی در نواحی ای که چگالی داده‌ها کم است یعنی دور از منیفولدها داده‌های کافی برای آموزش وجود ندارند. چون score matching بر اساس امیدریاضی روی داده‌های مشاهده شده کار می‌کند، در نواحی ای که هیچ داده‌ای وجود ندارد هیچ سیگنال آموزشی برای یادگیری تابع امتیاز وجود ندارد. این باعث می‌شود که تخمین تابع امتیاز در این نواحی بسیار نادقیق و اشتباه باشد. این تخمین‌های اشتباه باعث می‌شوند که فرآیند نمونه‌برداری Langevin Dynamics که قرار است از نواحی کم‌چگالی عبور کند تا به مدهای مختلف برسد دچار مشکل شود و نتواند به درستی همگرا شود.

(ب)

حل توسط NCSN: یکی از ایده‌های اصلی این است که داده‌ها را با سطوح مختلفی از نویز گوسی $\{\sigma_i\}_{i=1}^L$ آلوده کنیم و یک شبکه عصبی واحد آموزش دهیم که امتیاز تمام این توزیع‌های نویزی را تخمین بزند $s_\theta(x, \sigma)$. حل مشکل منیفولد به اینگونه است که از آنجا توزیع نویز گوسی در تمام فضا پخش می‌شود، وقتی به داده‌ها اضافه شود، تکیه‌گاه توزیع نویزی کل فضا خواهد بود و نه فقط یک منیفولد کم‌بعد. این باعث می‌شود تابع امتیاز در همه جا به خوبی تعریف شده باشد و مشکل ناسازگاری Score Matching حل شود. حل مشکل نواحی کم‌چگالی اینگونه است که نویزهای بزرگ با σ بالا باعث می‌شوند که داده‌ها در نواحی کم‌چگالی فضای اصلی پخش شوند و این نواحی را پر کنند. این کار سیگنال آموزشی لازم را برای Score Matching فراهم می‌کند تا تخمین امتیاز در آن نواحی نیز دقیق شود.

Annealed Langevin Dynamics: این روش نمونه‌برداری از نویزهای بزرگ شروع کرده و به تدریج نویز را کاهش می‌دهد. ابتدا با استفاده از σ_1 یعنی بزرگترین نویز نمونه‌هایی تولید می‌شوند. چون در این سطح نویز مدهای توزیع به هم متصل شده‌اند و نواحی کم‌چگالی پر شده‌اند، حرکت بین مدها آسان است و

نمونه‌برداری به خوبی انجام می‌شود. سپس این نمونه‌ها به عنوان نقطه شروع برای مرحله بعد یا نویز کمتر استفاده می‌شوند. این فرآیند ادامه می‌یابد تا به کوچکترین نویز σ_L برسیم. در اینجا بزرگترین نویز باید به اندازه‌ای بزرگ باشد که تمام مشکلات منیفولد و نواحی کم‌چگالی را پوشش دهد. نقش آن این است که مدهای جدا از هم توزیع را به هم وصل کند تا Langevin Dynamics بتواند آزادانه بین آن‌ها حرکت کند و توزیع کلی را به درستی یاد بگیرد. اگر بزرگترین نویز کوچک باشد مدل باز هم در مدهای محلی گیر می‌کند. همچنین کوچکترین نویز باید به اندازه‌ای کوچک باشد که عملاً تاثیری روی داده‌ها نگذارد و برای چشم انسان قابل تشخیص نباشد. درواقع نقش آن تضمین کیفیت نهایی و شفافیت تصویر است تا نمونه‌های تولیدشده تا حد ممکن به توزیع داده‌های واقعی نزدیک باشد.

بخش دوم: سوالات پیاده‌سازی

زیربخش اول: مدل پایه

۱- آماده‌سازی داده‌ها:

پس از نرمالایز کردن تصاویر و تشکیل داده‌های ترین و تست دنباله هندسی را ایجاد می‌کنیم که به صورت زیر است:

```
Dataset Loaded Successfully.
Train size: 60000, Test size: 10000
Data Range: [-1.00, 1.00]

Generated Noise Levels (Sigmas):
tensor([3.0000e+01, 1.2325e+01, 5.0632e+00, 2.0801e+00, 8.5454e-01, 3.5106e-01,
        1.4422e-01, 5.9251e-02, 2.4341e-02, 1.0000e-02])
```

شکل ۱. دنباله هندسی تشکیل شده

۲- معماری شبکه:

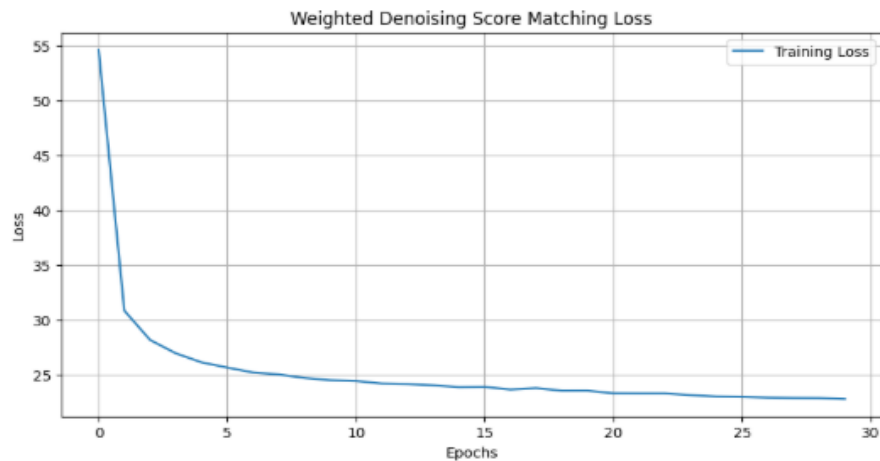
بر اساس مکانیزم FiLM و سایر مطالب گفته‌شده و همچنین جدول شماره ۲ به پیاده‌سازی معماری خواسته شده می‌پردازیم. خلاصه‌ای از معماری پیاده‌شده به صورت زیر است:

ScoreNet Model Summary					
Layer (Type:depth-idx)	Input Shape	Output Shape	Param #	Kernel Shape	Mult-Adds
ScoreNet	[4, 1, 28, 28]	[4, 1, 28, 28]	--	--	--
Sequential: 1-1	[4]	[4, 256]	--	--	--
Linear: 2-2	[4]	[4, 256]	(128)	--	--
Linear: 2-3	[4, 256]	[4, 256]	65,792	--	263,168
Linear: 2-4	[4, 256]	[4, 256]	--	--	--
Conv2d: 2-5	[4, 1, 28, 28]	[4, 64, 28, 28]	65,792	[3, 3]	263,168
AdaptiveResBlock: 1-3	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	2,067,640
Conv2d: 2-5	[4, 64, 28, 28]	[4, 64, 28, 28]	36,928	[3, 3]	115,886,208
FiLM: 2-6	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	131,584
Linear: 3-1	[4, 256]	[4, 128]	32,896	--	512
GroupNorm: 3-2	[4, 64, 28, 28]	[4, 64, 28, 28]	128	--	--
Linear: 3-3	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
Conv2d: 2-8	[4, 64, 28, 28]	[4, 64, 28, 28]	36,928	[3, 3]	115,886,208
FiLM: 2-9	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
Linear: 3-3	[4, 256]	[4, 128]	32,896	--	131,584
GroupNorm: 3-4	[4, 64, 28, 28]	[4, 64, 28, 28]	128	--	512
Linear: 2-10	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
Identity: 2-11	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
AdaptiveResBlock: 1-5	[4, 64, 14, 14]	[4, 128, 14, 14]	--	--	2
Conv2d: 2-12	[4, 64, 14, 14]	[4, 128, 14, 14]	73,856	[3, 3]	57,903,104
FiLM: 2-13	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Linear: 3-5	[4, 256]	[4, 256]	65,792	--	263,168
GroupNorm: 3-6	[4, 128, 14, 14]	[4, 128, 14, 14]	256	--	1,024
Linear: 3-7	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Conv2d: 2-15	[4, 128, 14, 14]	[4, 128, 14, 14]	147,584	[3, 3]	115,795,856
FiLM: 2-16	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Linear: 3-7	[4, 256]	[4, 256]	65,792	--	263,168
GroupNorm: 3-8	[4, 128, 14, 14]	[4, 128, 14, 14]	256	--	1,024
Linear: 3-8	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Conv2d: 2-18	[4, 64, 14, 14]	[4, 128, 14, 14]	8,320	[1, 1]	6,522,880
AdaptiveResBlock: 1-6	[4, 128, 7, 7]	[4, 256, 7, 7]	--	--	2
Conv2d: 2-19	[4, 128, 7, 7]	[4, 256, 7, 7]	295,168	[3, 3]	57,852,928
FiLM: 2-20	[4, 256, 7, 7]	[4, 256, 7, 7]	--	--	263,336
Linear: 3-9	[4, 256]	[4, 512]	131,584	--	2,048
GroupNorm: 3-10	[4, 256, 7, 7]	[4, 256, 7, 7]	512	--	115,655,680
Linear: 2-21	[4, 256, 7, 7]	[4, 256, 7, 7]	--	--	--
Conv2d: 2-22	[4, 256, 7, 7]	[4, 256, 7, 7]	900,000	[3, 3]	--
FiLM: 2-23	[4, 256, 7, 7]	[4, 256, 7, 7]	--	--	--
Linear: 3-11	[4, 256]	[4, 512]	131,584	--	526,336
GroupNorm: 3-12	[4, 256, 7, 7]	[4, 256, 7, 7]	512	--	2,048
Linear: 2-24	[4, 256, 7, 7]	[4, 256, 7, 7]	--	--	--
Conv2d: 2-25	[4, 128, 7, 7]	[4, 256, 7, 7]	33,024	[1, 1]	6,472,704
AdaptiveResBlock: 1-8	[4, 384, 14, 14]	[4, 128, 14, 14]	--	--	--
Conv2d: 2-26	[4, 384, 14, 14]	[4, 128, 14, 14]	442,496	[3, 3]	346,916,864
FiLM: 2-27	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Linear: 3-13	[4, 256]	[4, 256]	65,792	--	263,168
GroupNorm: 3-14	[4, 128, 14, 14]	[4, 128, 14, 14]	256	--	1,024
Linear: 3-14	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Conv2d: 2-29	[4, 128, 14, 14]	[4, 128, 14, 14]	147,584	[3, 3]	115,795,856
FiLM: 2-30	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Linear: 3-15	[4, 256]	[4, 256]	65,792	--	263,168
GroupNorm: 3-16	[4, 128, 14, 14]	[4, 128, 14, 14]	256	--	1,024
Linear: 2-31	[4, 128, 14, 14]	[4, 128, 14, 14]	--	--	--
Conv2d: 2-32	[4, 384, 14, 14]	[4, 128, 14, 14]	49,280	[1, 1]	38,635,520
AdaptiveResBlock: 1-9	[4, 192, 28, 28]	[4, 64, 28, 28]	--	--	--
Conv2d: 2-33	[4, 192, 28, 28]	[4, 64, 28, 28]	110,656	[3, 3]	347,017,216
FiLM: 2-34	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
Linear: 3-17	[4, 256]	[4, 128]	32,896	--	131,584
GroupNorm: 3-18	[4, 64, 28, 28]	[4, 64, 28, 28]	128	--	512
Linear: 2-35	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
Conv2d: 2-36	[4, 64, 28, 28]	[4, 64, 28, 28]	36,928	[3, 3]	115,886,208
FiLM: 2-37	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
Linear: 3-19	[4, 256]	[4, 128]	32,896	--	131,584
GroupNorm: 3-20	[4, 64, 28, 28]	[4, 64, 28, 28]	128	--	512
Linear: 2-38	[4, 64, 28, 28]	[4, 64, 28, 28]	--	--	--
Conv2d: 2-39	[4, 192, 28, 28]	[4, 64, 28, 28]	12,352	[1, 1]	38,735,672
Conv2d: 1-10	[4, 64, 28, 28]	[4, 1, 28, 28]	577	[3, 3]	1,809,472
Total params: 2,814,593					
Trainable params: 2,814,465					
Non-trainable params: 128					
Total mult-adds (units: GIGABYTES): 1.60					
Input size (MB): 0.01					
Forward/backward pass size (MB): 26.22					
Param size (MB): 11.26					
Estimated Total Size (MB): 37.49					

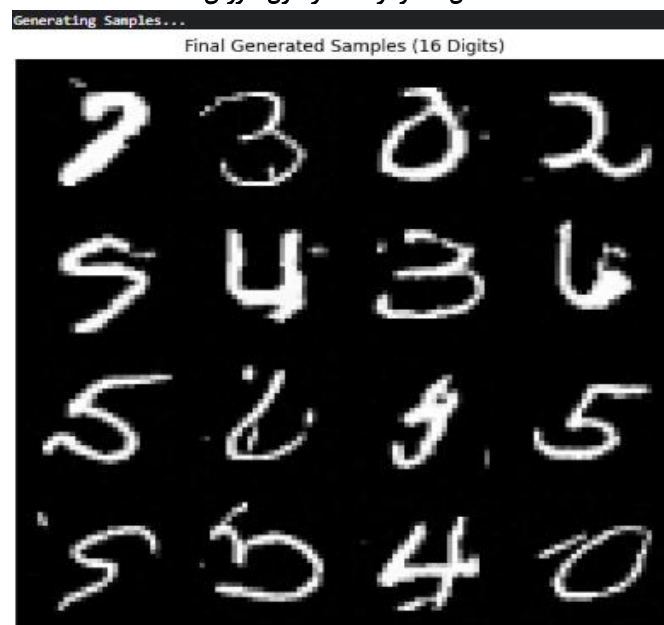
شکل ۲. خلاصه مدل پیاده‌سازی شده

۳- آموزش و خروجی‌ها:

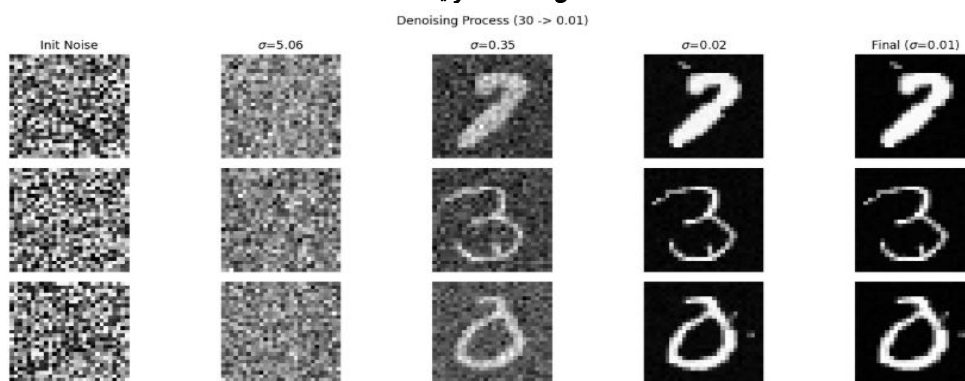
مدل را با تابع هزینه گفته‌شده آموزش می‌دهیم و الگوریتم نمونه‌برداری Annealed Langevin Dynamics را پیاده‌سازی می‌کنیم. نتایج خواسته‌ها به شرح زیر است:



شکل ۳. نمودار خطا در طول آموزش



شکل ۴. Grid تولیدشده



شکل ۵. روند تبدیل نویز خالص به عدد نهایی برای سه نمونه

```
SIGMA_BEGIN = 30  
SIGMA_END = 0.01  
NUM_LEVELS = 10  
BATCH_SIZE = 64  
LEARNING_RATE = 2e-4  
EPOCHS = 30  
LANGEVIN_STEPS = 150  
STEP_LR = 2e-5
```

شکل ۶. هایپرپارامترهای استفاده شده در آموزش

افت سریع نمودار خطا و سپس افت کمتر نشان دهنده این است که مدل ویژگی‌های کلی مثل اینکه تصویر باید سیاه باشد و لکه سفید وسط تصویر است را به سرعت یاد گرفته اما در یادگیری منیفولد دقیق داده‌ها یعنی اینکه لبه‌ها دقیقا کجا هستند و پیوستگی خطوط دچار مشکل شده یا در یکی از لوکال مینیمم‌ها گیر کرده است. از نمونه‌های تولید شده نیز می‌توان مشاهده کرد که گاهی مشکل عدم پیوستگی در ارقام تولید شده وجود دارد و همچنین می‌توان نتیجه گرفت که مدل بیشتر روی لبه‌ها تمرکز کرده تا میزان ضخامت. این نشان دهنده این است که در نمونه برداری زنجیره مارکوف نتوانسته به خوبی همگرا شود که به این معناست که احتمالا یا تعداد استپ‌ها کم بوده و یا سایز استپ‌ها باید تغییر کند. همچنین فرآیند نويززدایی نشان می‌دهد که منطق کلی مدل درست کار می‌کند و جهت گرادیان‌ها صحیح هستند اما در مراحل پایانی دینویزینگ مدل قدرت لازم برای صاف یا پر کردن نویزهای ریز باقی مانده را ندارد.

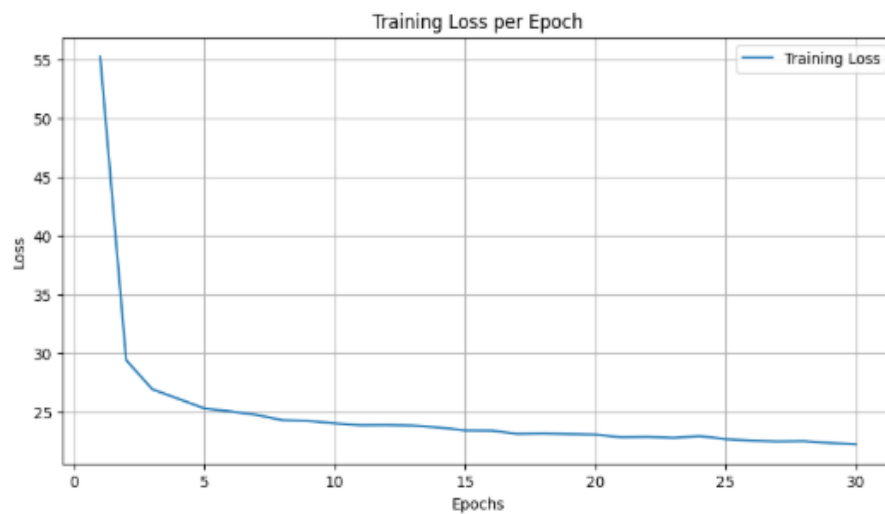
زیربخش دوم: مدل شرطی

تغییرات لازم برای پیاده‌سازی را لحاظ می‌کنیم. همچنین مهم‌ترین تکنیک آموزشی در مقاله استفاده از EMA است. در این تکنیک به جای استفاده از آخرین وزن‌های مدل از یک تاریخچه میانگیری شده وزن‌ها استفاده می‌کنیم. در میانگین گیری نمایی وزن‌های آخر تاثیر بیشتری دارند و وزن‌های قدیمی به مرور زمان تاثیرشان به صورت نمایی کم می‌شود و به صفر میل می‌کند. با استفاده از این تکنیک وزن‌ها نرم‌تر پایدارتر و دقیق‌تر می‌شوند.

خلاصه‌ای از مدل جدید با تغییرات پیاده‌سازی گفته شده در صفحه بعد آمده است:

MODEL SUMMARY				
Layer (type:depth-idx)	Input Shape	Output Shape	Param #	Kernel Shape
ConditionalScoreNet	[1, 1, 28, 28]	[1, 1, 28, 28]	--	--
└Sequential: 1-1	[1]	[1, 256]	--	--
└GaussianFourierProjection: 2-1	[1]	[1, 256]	(128)	--
└Linear: 2-2	[1, 256]	[1, 256]	65,792	--
└SiLU: 2-3	[1, 256]	[1, 256]	--	--
└Linear: 2-4	[1, 256]	[1, 256]	65,792	--
└Embedding: 1-2	[1]	[1, 256]	2,560	--
└Conv2d: 1-3	[1, 1, 28, 28]	[1, 64, 28, 28]	640	[3, 3]
└AdaptiveResBlock: 1-4	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Conv2d: 2-5	[1, 64, 28, 28]	[1, 64, 28, 28]	36,928	[3, 3]
└FiLM: 2-6	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Linear: 3-1	[1, 256]	[1, 128]	32,896	--
└GroupNorm: 3-2	[1, 64, 28, 28]	[1, 64, 28, 28]	128	--
└SiLU: 2-7	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Conv2d: 2-8	[1, 64, 28, 28]	[1, 64, 28, 28]	36,928	[3, 3]
└FiLM: 2-9	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Linear: 3-3	[1, 256]	[1, 128]	32,896	--
└GroupNorm: 3-4	[1, 64, 28, 28]	[1, 64, 28, 28]	128	--
└SiLU: 2-10	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Identity: 2-11	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└AvgPool2d: 1-5	[1, 64, 28, 28]	[1, 64, 14, 14]	--	2
└AdaptiveResBlock: 1-6	[1, 64, 14, 14]	[1, 128, 14, 14]	--	--
└Conv2d: 2-12	[1, 64, 14, 14]	[1, 128, 14, 14]	73,856	[3, 3]
└FiLM: 2-13	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Linear: 3-5	[1, 256]	[1, 256]	65,792	--
└GroupNorm: 3-6	[1, 128, 14, 14]	[1, 128, 14, 14]	256	--
└SiLU: 2-14	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Conv2d: 2-15	[1, 128, 14, 14]	[1, 128, 14, 14]	147,584	[3, 3]
└FiLM: 2-16	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Linear: 3-7	[1, 256]	[1, 256]	65,792	--
└GroupNorm: 3-8	[1, 128, 14, 14]	[1, 128, 14, 14]	256	--
└SiLU: 2-17	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Conv2d: 2-18	[1, 64, 14, 14]	[1, 128, 14, 14]	8,320	[1, 1]
└AvgPool2d: 1-7	[1, 128, 14, 14]	[1, 128, 7, 7]	--	2
└AdaptiveResBlock: 1-8	[1, 128, 7, 7]	[1, 256, 7, 7]	--	--
└Conv2d: 2-19	[1, 128, 7, 7]	[1, 256, 7, 7]	295,168	[3, 3]
└FiLM: 2-20	[1, 256, 7, 7]	[1, 256, 7, 7]	--	--
└Linear: 3-9	[1, 256]	[1, 512]	131,584	--
└GroupNorm: 3-10	[1, 256, 7, 7]	[1, 256, 7, 7]	512	--
└SiLU: 2-21	[1, 256, 7, 7]	[1, 256, 7, 7]	--	--
└Conv2d: 2-22	[1, 256, 7, 7]	[1, 256, 7, 7]	590,880	[3, 3]
└FiLM: 2-23	[1, 256, 7, 7]	[1, 256, 7, 7]	--	--
└Linear: 3-11	[1, 256]	[1, 512]	131,584	--
└GroupNorm: 3-12	[1, 256, 7, 7]	[1, 256, 7, 7]	512	--
└SiLU: 2-24	[1, 256, 7, 7]	[1, 256, 7, 7]	--	--
└Conv2d: 2-25	[1, 128, 7, 7]	[1, 256, 7, 7]	33,024	[1, 1]
└AdaptiveResBlock: 1-9	[1, 384, 14, 14]	[1, 128, 14, 14]	--	--
└Conv2d: 2-26	[1, 384, 14, 14]	[1, 128, 14, 14]	442,496	[3, 3]
└FiLM: 2-27	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Linear: 3-13	[1, 256]	[1, 256]	65,792	--
└GroupNorm: 3-14	[1, 128, 14, 14]	[1, 128, 14, 14]	256	--
└SiLU: 2-28	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Conv2d: 2-29	[1, 128, 14, 14]	[1, 128, 14, 14]	147,584	[3, 3]
└FiLM: 2-30	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Linear: 3-15	[1, 256]	[1, 256]	65,792	--
└GroupNorm: 3-16	[1, 128, 14, 14]	[1, 128, 14, 14]	256	--
└SiLU: 2-31	[1, 128, 14, 14]	[1, 128, 14, 14]	--	--
└Conv2d: 2-32	[1, 384, 14, 14]	[1, 128, 14, 14]	49,280	[1, 1]
└AdaptiveResBlock: 1-10	[1, 192, 28, 28]	[1, 64, 28, 28]	--	--
└Conv2d: 2-33	[1, 192, 28, 28]	[1, 64, 28, 28]	110,656	[3, 3]
└FiLM: 2-34	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Linear: 3-17	[1, 256]	[1, 128]	32,896	--
└GroupNorm: 3-18	[1, 64, 28, 28]	[1, 64, 28, 28]	128	--
└SiLU: 2-35	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Conv2d: 2-36	[1, 64, 28, 28]	[1, 64, 28, 28]	36,928	[3, 3]
└FiLM: 2-37	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Linear: 3-19	[1, 256]	[1, 128]	32,896	--
└GroupNorm: 3-20	[1, 64, 28, 28]	[1, 64, 28, 28]	128	--
└SiLU: 2-38	[1, 64, 28, 28]	[1, 64, 28, 28]	--	--
└Conv2d: 2-39	[1, 192, 28, 28]	[1, 64, 28, 28]	12,352	[1, 1]
└Conv2d: 1-11	[1, 64, 28, 28]	[1, 1, 28, 28]	577	[3, 3]
Total params: 2,817,153				
Trainable params: 2,817,025				
Non-trainable params: 128				
Total mult-adds (Units.MEGABYTES): 400.38				
Input size (MB): 0.00				
Forward/backward pass size (MB): 6.56				
Params size (MB): 11.27				
Estimated Total Size (MB): 17.83				

شکل ۷. خلاصه‌ای از پیاده‌سازی مدل جدید



شکل ۸. نمودار خطا در حین آموزش



شکل ۹. Grid تولید شده

همانطور که مشاهده می‌شود در اغلب سمپل‌های این بخش نسبت به بخش قبل پیشرفت اتفاق افتاده. در اینجا اعداد از هم تا حد خوبی قابل تفکیک هستند اما در قبلی گاهی این چنین نبود. همچنین خطوط در این مدل پیوستگی و یکپارچگی بهتری دارند. افزون بر آن ضخامت اعداد در اینجا طبیعی‌تر است و بیش از حد نازک نیستند. پس زمینه نیز در اینجا تمیزتر و بدون نویزهای ریز است و در مجموع می‌توان اظهار کرد که مدل واقعا بهتر شده است.

همچنین اگر بخواهیم خود این نتایج را جداگانه بررسی کنیم می‌توان گفت که شرطی‌سازی تا حد خوبی موفقیت آمیز بوده و بیشتر داده‌های هر ردیف از کلاس موردنظر بوده‌اند هرچند که خطا نیز وجود دارد. در اعداد با پیچیدگی کمتر کیفیت بالاتر بوده و در اعداد با پیچیدگی‌های بالاتر خطا نیز بالاتر رفته‌است.